

Creative Technology Solutions

CS 230 Project Software Design Template

Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 6

Document Revision History

Version	Date	Author	Comments
1.0	1/21/2025	Filip Arghir	Initial Draft of Doc
2.0	2/5/2025	Filip Arghir	Evaluation and Recommendations

Executive Summary

The Gaming Room aims to develop a web-based version of their current game Draw It or Lose It which is currently Android only. They would like to know how to set up the environment and facilitate the development of the web-based version of their game. This document will address their software requirements and hardware requirements as well as a progress review.

Requirements

Some requirements for this project are that the game needs to be able to support teams and those teams need to have multiple players as well. There must also be unique names for the teams and players can check whether or not a name is in use. There can also only be one instance of the game that can exist in memory at a time and therefore creating unique identifier for each instance of a game team or player would need to be created.

Design Constraints

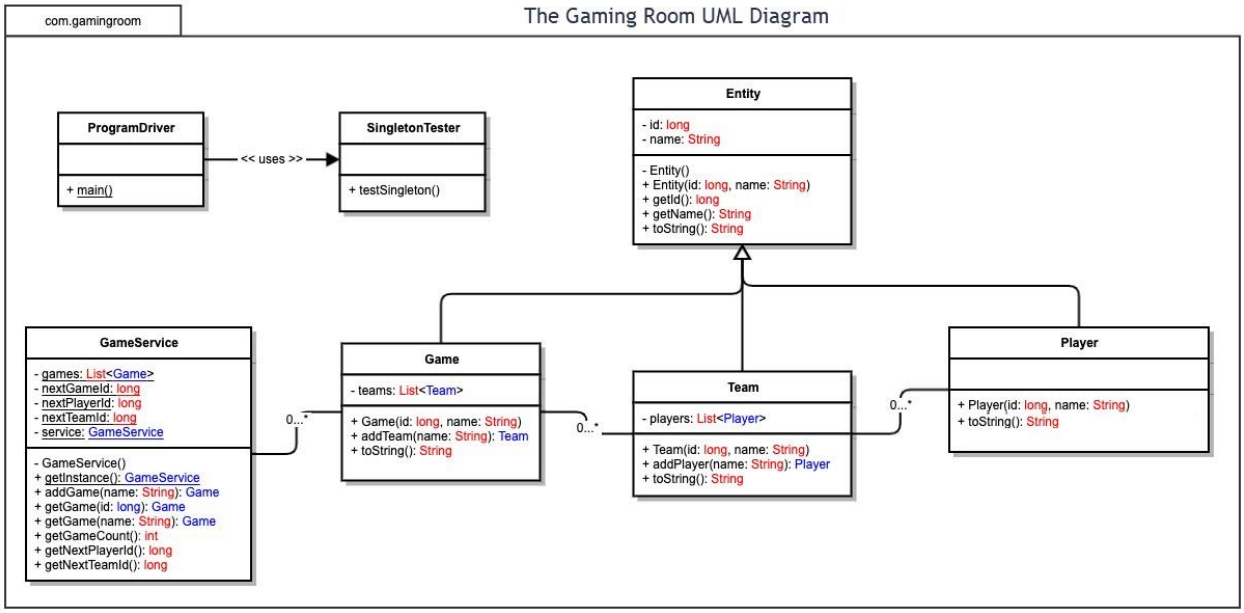
Developing this game into a web-based format does cause some restrictions. Specifically, platform compatibility is accessible across multiple systems and forms of devices. There needs to be security to prevent the system from being breached and from unauthorized access. The game also needs to be able to handle several users easily and at a stable rate without causing high latency.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

This UML model for The Gaming Room represents the key entities and their relationships within the system and provides a framework for how different components interact. The UML model below describes the encapsulation using attributes stored in private fields with public getters and inheritance is also viewed through the Game, Team, and Player classes as they are all inherited from the Entity class. The Entity class is a base class and contains common attributes such as id and name. The GameService class manages the game instances using a singleton pattern. Game Class contains a list of teams and methods to add new teams. Team Class contains a list of players and methods to add players. Player class represents an individual player with their own unique attributes.



EvaluationS

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Higher Costs compared to others, Scalable but less customizable than others, very secure as well. Limited hosting options	Lower costs compared to windows and Mac, scalable and fully customizable, can be very secure. Extensive server support	Higher licensing costs compared to Linux not as much as Mac, scalable and customizable. GUI based management.	Mobile devices are not great for hosting servers but good for client-side access-cloud hosting servers also do exist.
Client Side	Safari optimization and a deep understanding of MacOS Safari compatibility also needs to exist and Apple developer fees exist as well.	Requires compatibility with several distributions. Open-source tools do exist therefore keeping costs low.	Edge and Chrome are requirements for compatibility and development costs are centered around Visual Studio. Several tools do exist and therefore compatibility is also somewhat broad.	Good web design and mobile friendly UI. Android and iOS needs native optimization therefore developers need to understand both and Apple Fees still exist here as well.
Development Tools	Xcode, Swift, react, Node.js	VS code, Eclipse, Python, Node.js	Visual Studio, Unity, Javas, C# - (I have personally used Unity for a Web based game before)	Android Studio, Xcode, React, Flutter.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**:

I would suggest either Linux for server-side hosting due to cost effectiveness and the fact that Linux really does not require much to function as a unit and has great support for web-based applications. For client sided work windows might be considered for development and possible testing of applications as most clients use windows and therefore Windows sets some basic standards.

2. **Operating Systems Architectures**:

For operating system architectures Linux provides better performance, scalability and security for web applications and once again Windows would be great for using client-side compatibility and testing. Linux allows for customization for server needs and therefore if several clients will connect to our web-based game having a Linux based server will allow more clients to connect since more hardware can be dedicated to hosting the server.

3. **Storage Management**:

Linux based local storage is a possible selection since we are already hosting the server. However, this can raise costs as we are storing the information, and more clients would be connecting and pulling more information. Therefore, possible cloud-based storage can be suggested but costs would be higher.

4. **Memory Management**:

Linux as an operating system already pulls very little from hardware and virtual memory management can help with our performance and therefore allow for smoother gameplay even with a steady rate for drawing that are rendered. Adding more memory but with Node.js we can perform Garbage Collection and manage our memory properly and therefore free up hardware. Windows also has memory management which implements virtual memory and segmentation which allows the system to run big applications, but Linux already uses less as a whole and therefore incorporating web tokens we can keep using Linux.

5. **Distributed Systems and Networks**:

RESTful API would be great for communication between the web-based server and clients across several platforms. This system already incorporates HTTP protocols and allows easy scalability since RESTful APIs are stateless and therefore more servers could hypothetically be added and are compatible with any platform as long as there are HTTP requests. RESTful API can use Apache and store game data as well if needed and can also use docker for containerization! For Client side all they need to do is connect via web browser and clients send HTTP requests and our server would process said requests and update the game state.

6. **Security**:

For security using HTTPS and SSL/TLS to secure data transmission would be suggested and implements secure user authentication to prevent an overload of connections and destabilizing the server. OAuth would also be great for ensuring that there is proper user authentication and token authorization so real users can join the game. Having OAuth would also allow for securing player data for several platforms and therefore it would also be plausible for a ranked system or cross platform progression for player data.